

Multithreaded programming

Procesos

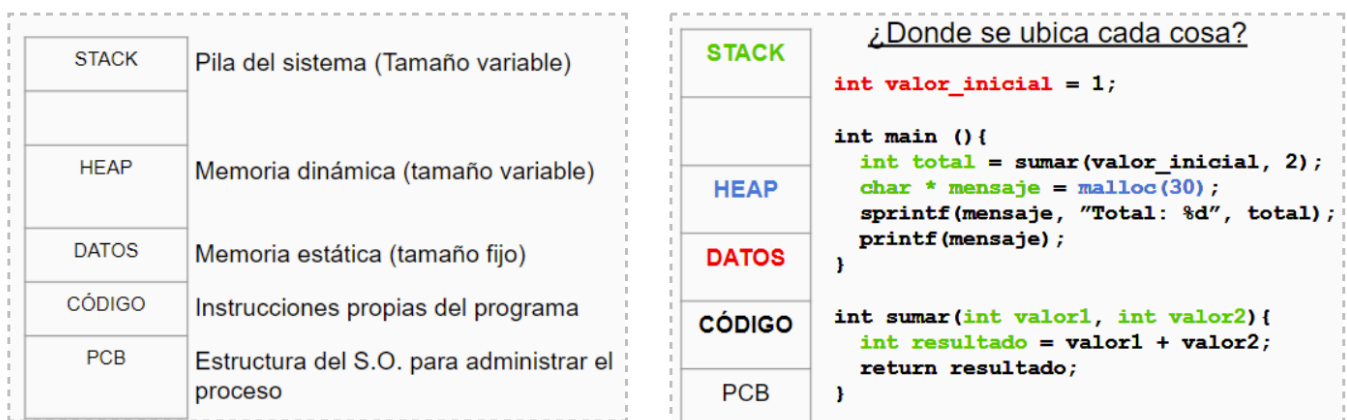
Es un programa que está siendo ejecutado en un instante determinado y, además, se le asignó memoria o recursos para que pueda funcionar. También puede decirse que un proceso es una instancia de un programa. Los procesos son la unidad de trabajo de un SO. Están formados por:

- Secuencia de instrucciones que deben ejecutar el procesador
- Conjunto de datos.
- Estado (nos da la información precisa de lo que está haciendo el proceso en un momento determinado).
- Atributos que lo identifican.
- Tiene los recursos que le fueron asignados.

Entonces un proceso es toda la estructura que permite que un conjunto de instrucciones se pueda ejecutar. Todo proceso debe estar representado en algún lugar, este lugar es la memoria RAM.

Entorno de un proceso: conjunto de variables que se le pasan al proceso al momento de su ejecución.

Estructura de un proceso



- **Código:** espacio asignado para almacenar la secuencia de instrucciones del programa. Las instrucciones (así como están, en lenguaje de máquina) se cargan del disco a la memoria. **Los procesos no lo pueden modificar**
- **Datos:** espacio asignado para almacenar las variables globales. Todas las variables que están definidas por fuera de la función main van acá. **No lo pueden modificar el proceso.**
- **Stack (memoria estática):** Es memoria que se reserva al declarar variables de cualquier tipo de dato, así como también punteros a tipos de datos (hay que aclarar que se guarda la *declaración* del puntero, no el dato al que referencia. Ese dato se guarda en el HEAP). El stack es una estructura en forma de pila y el programador no deberá encargarse de liberarla.
 - Variables locales (locales a la función que se está ejecutando). Si la variable está inicializada también se guarda el valor.

- Llamadas a funciones que el programador va haciendo. Al producirse una llamada a una función, se almacena en el stack la dirección a la que debe retornar la ejecución tras finalizar la llamada. Una vez que se cambia el contexto de ejecución se borran las variables.
- Parámetros.
- Retorno de las funciones. **Los procesos pueden modificar el stack**
- *Heap* (memoria dinámica): memoria que se reserva en tiempo de ejecución. Se va reservando a medida que es necesaria, y se libera cuando se deja de utilizar. Es responsabilidad del programador liberarla cuando no se utilice más. Varía dependiendo de las necesidades del proceso, por lo tanto, **los procesos la puede modificar**. Algunas ventajas que ofrece la memoria dinámica frente a la memoria estática son:
 - Puede reservar espacio para variables de tamaño no conocido hasta el momento de ejecución.
 - Mientras se mantenga alguna referencia a los bloques de memoria, estos pueden sobrevivir al bloque de código que los creó.
- *PCB* (process control block): Es una estructura donde se guarda el contexto de ejecución de un proceso y es usada especialmente para la multiprogramación. Hay uno por cada proceso en el sistema. Es creado y gestionado por el SO. Contiene la información necesaria para que el SO administre el proceso y pueda guardar el contexto, en caso de un cambio de contexto solicitado por la CPU para ejecutar otra cosa. El procesador tiene el contexto del proceso que está ejecutándose en ese instante, entonces, si se quiere ejecutar un procesos que ya venía ejecutando entes, primero hay que cargar su PBC. **Los procesos no pueden modificarlo. El PCB está siempre cargado en RAM** y está compuesto por:
 - PSW: el estado del proceso
 - Identificador:
 - PID: Process ID.
 - PPID: Identificador del padre del proceso.
 - UID: Identificador del usuario que inició el proceso.
 - IP/PC
 - Registros del procesador
 - Información de planificación de CPU (prioridad de ejecución de un proceso)
 - Información de manejo de memoria
 - Información de E/S
 - Información contable (como el tiempo de CPU)

Tiempo de vida de los datos

Todos los datos tienen un tiempo de vida. En C hay tres tipos de duración:

- Estática: son variables que se crean una única vez junto con la creación del proceso y se destruyen junto con la destrucción del mismo. Son únicas y generalmente pueden ser utilizadas desde cualquier programa. Para generar una variable estática se puede declarar fuera de la función principal, o bien usando el calificador `static`.
- Automática: son variables locales que no son declaradas de forma estática. Se crean al entrar al bloque en el que fueron declaradas y se destruyen al salir de él.
- Asignada: es la memoria que se reserva de forma dinámica (en el heap).

Ciclo de vida de un proceso

El ciclo de vida de un proceso es el tiempo que transcurre entre la creación y finalización de un proceso. Durante este lapso, el proceso pasa por varios estados.

Estados de un proceso

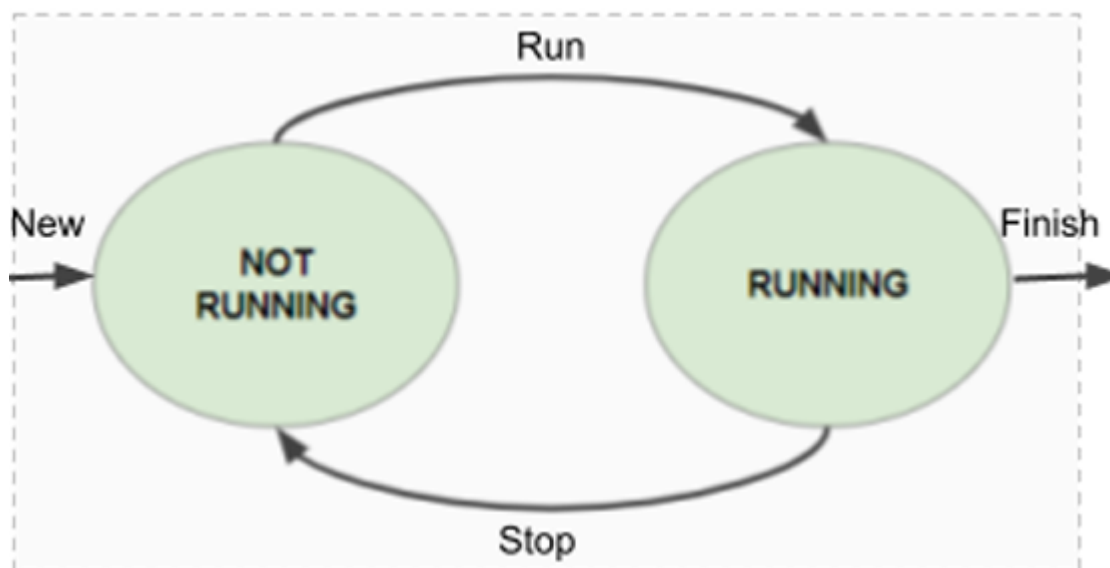
El estado de un proceso es el comportamiento del mismo. Conocer su estado nos permite entender en qué condición se encuentra.

Llamadas al sistemas bloqueantes o no bloqueantes

- **Bloqueantes:** se ejecutan en el momento en que se llaman. Hacen que un proceso se bloquee.
- **No bloqueantes:** el proceso, una vez que ejecuta la syscall, no necesariamente pasa al estado de bloqueado. El proceso no sale de running, salvo cuando termina de ejecutar. En general, la syscall va a ser no bloqueante cuando un proceso no la necesite urgentemente para seguir ejecutando.
-

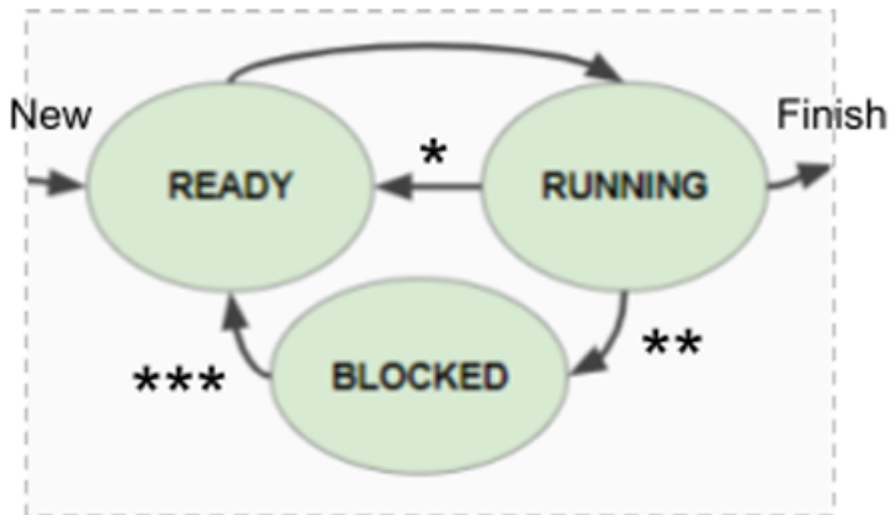
	Bloqueante	No bloqueante
Operación se puede realizar inmediatamente	Realiza la operación	Realiza la operación
Operación va a tardar mucho (indefinido)	Bloquea el proceso. Sigue con otros procesos que tardan menos	• No realiza la operación en <u>ese</u> momento. • Continúa ejecutando.
Valores de retorno	Ok/Error	Ok/Error/Reintentar

Diagrama de dos estados



Running/Ejecutando: Un proceso que está usando la CPU **Not running/No ejecutando:** Un proceso (o muchos) que está esperando usar la CPU

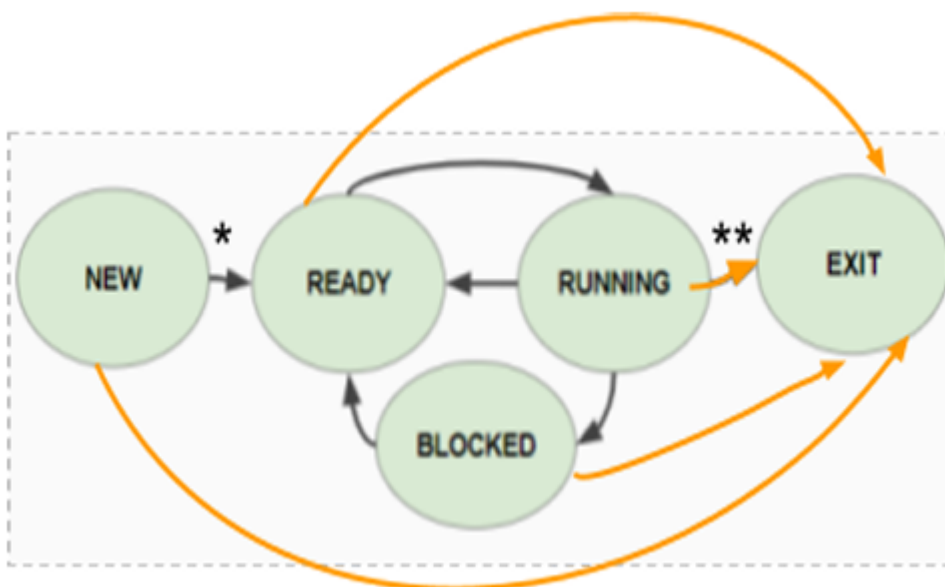
Diagrama de tres estados



El estado *not running* se divide en 2:

- **Ready/Listos:** es una cola que contienen todos los procesos que están listos para ejecutar, es decir, que son elegibles por el procesador. Ya deben tener sus registros (PCB) en memoria. El dispatcher es quien se encarga de mandar los procesos a running *Puede ser que un proceso vuelva de Running a Ready sin bloquearse si simplemente se le acabó el tiempo que tenía designado para ejecutar (interrupción de clock).
- **Blocked:** son procesos que ya ejecutaron en la CPU, pero que no pueden ser asignados al procesador porque están esperando que ocurra algún tipo de evento. Un proceso que está bloqueado nunca puede pasar a running directamente, si no que antes debe pasar a ready y ser elegido por el SO para ejecutar nuevamente. **Cuando un proceso pasa de running a bloqueado es porque hizo una llamada a sistema bloqueante. ***Para que un proceso pase de bloqueado a ready, debe avisarle al SO que el evento que estaba esperando ya finalizó. Esto se informa mediante una interrupción.

Diagrama de cinco estados



- **New/Nuevo:** Cuando se quiere ejecutar un programa, pasa un tiempo entre que el CPU recibe el mensaje de ejecutar y cuando realmente se empieza a ejecutar.

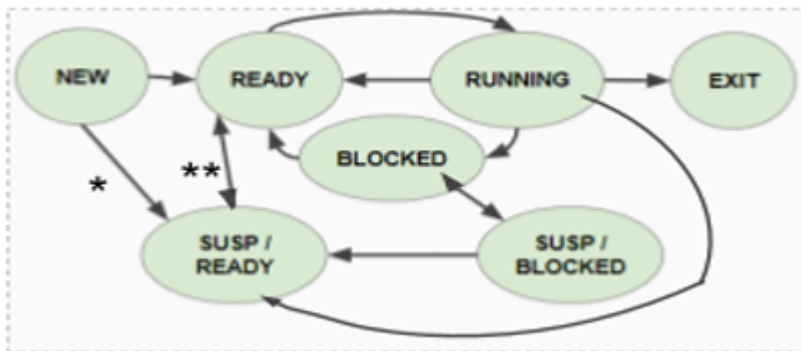
- Se preparan las estructuras
- Se inicializa el PCB
- Se espera la admisión

*Una vez que terminan estas tareas el proceso pasa a ready

- **Exit/Finalizado:** De cualquier estado puede pasarse al estado exit. Acá se libera la memoria, recursos, etc.
 - Valor de retorno: es importante que cuando un proceso finaliza guarde su valor de retorno por si otro proceso quiere consultarlo.
 - Todas las estructuras menos el PCB se eliminan. El PCB se almacena para fines estadísticos/contables.

**Pueden pasar por tres motivos: porque ejecutó su última instrucción o por un error (que puede traducirse en una interrupción), o porque el proceso se mate a sí mismo (kill)

Diagrama de siete estados



Desde cualquier estado se puede pasar a exit. Se agrega la idea de suspender un proceso. Hoy en día no tiene mucho sentido pero antes se usaba para pasar las estructuras de un proceso al disco para liberar memoria y poder ejecutar más procesos.

- **Susp/Ready:** la diferencia con ready es que estos se encuentran en el disco y deberán ser pasados a memoria para ejecutar.

*Un proceso pasa de new a susp/ready cuando hay demasiados procesos en ready. Esto es porque ready está en memoria principal y susp/ready en disco. De todas formas el pcb del proceso se pasa a ready y si se decide ejecutarlo se irá a buscarlo al disco. **Cuando el nivel de multiprogramación disminuye y se decide traer un proceso que estaba en disco a RAM o, caso contrario, aumenta el nivel de multiprogramación y se decide mandar al disco a un proceso.

- **Susp/Blocked:** Se toman los procesos que están bloqueados y se pasan sus estructuras (menos el PCB) al disco

Estados en Linux

- R (runnable/running): es la conjunción del estado running y ready.
- S (sleep): es un blocked "interrumpible".
- Super bloqueado: no se puede interrumpir.

- T:stopped.
- +:me indica que está en el foreground.
- Z(zombie): estado exit. El proceso queda en este estado hasta que alguien lo reclama. El proceso terminó de ejecutar pero el PCB sigue estando en memoria.
- D (uninterruptable sleep).
- X (dead).

Creación de procesos

Formas de creación de un proceso:

- Lo crea el SO para dar algún servicio.
- Lo crea otro proceso.
 - El proceso que solicita crearlo se llama padre y al proceso creado se lo llama hijo
 - El proceso padre e hijo puede ejecutar de forma concurrente o el padre puede esperar a que los hijos finalicen. El proceso padre y el hijo pueden ser dos procesos completamente distintos.

Pasos para crear un proceso:

1. Asignar PID.
2. Reservar espacio para estructuras
3. Inicializar PCB
4. Ubicar PCB en listas de planificación

Creación de un proceso en Linux: los procesos hijos se crean a través de la syscall `fork()` y se les asigna el valor 0. Lo que hace `fork()` es duplicar al proceso que lo llamó pero con un nuevo PID, agregándole el PPID (que es el PID del proceso padre) y los registros, pilas, heap, se copian igual. Incluso el PC se copia. Cuando un proceso padre crea un hijo, puede ocurrir que el padre siga ejecutando concurrentemente con su hijo o que espere a que termine o, incluso, pueden ejecutar distintas partes del programa. Puede ocurrir que el proceso sea la copia exacta del padre o que se le de una nueva imagen que reemplace la anterior (`execv`). En cuanto a los recursos, el hijo puede pedir al SO o puede estar restringido al uso de un subset de los del padre.

Tabla de procesos (TDP): cuando un proceso es creado, es agregado a la tabla de procesos del sistema. Es manejada por el SO y el grado de multiprogramación es definido a partir de la cantidad de procesos activos en la TDP.

Finalización de procesos

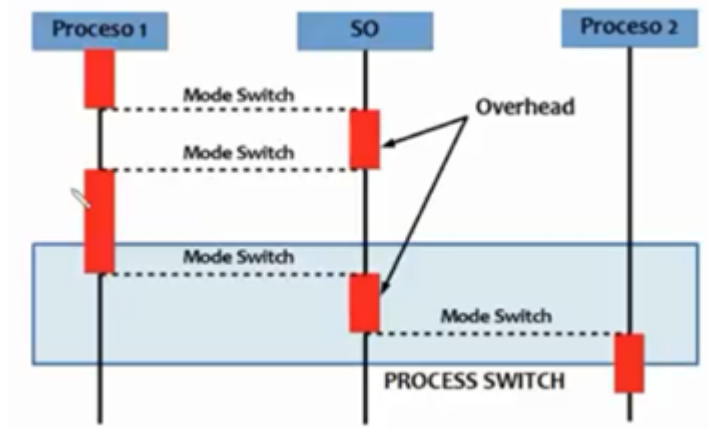
Formas de finalización de un proceso:

- Lo finaliza el SO (kill).
- Lo finaliza otro proceso (kill).
- Se finaliza el mismo:
 - Normal exit.
 - Abnormal exit.

Finalización de un proceso en Linux: un proceso le indica al SO que finaliza con la syscall `exit()`. Un proceso hijo le indica al padre que finaliza y su resultado con la syscall `wait()`. Un padre puede interrumpir la ejecución de sus hijos con la syscall `abort()`.

Cambio de procesos

Se produce cuando viene ejecutando un proceso en el procesador, por algún motivo deja de ejecutar y comienza a ejecutar otro. El cambio de contexto puede tener como objetivo ejecutar otro proceso, a tender una interrupción o ejecutar una syscall



Se debe guardar el contexto de ejecución del proceso 1 para luego poder reanudarlo desde que fue interrumpido.